Can AlgoTutor Change Attitudes toward Algorithms?

Jungsoon Yoo Sung Yoo Suk Seo Chrisila Pettey
Computer Science Department
Middle Tennessee State University
United States
01-615-898-2397
csyoojp@mtsu.edu

ABSTRACT
The ability to design an algorithm is one of the most important learning outcomes of a computer science program. Unfortunately, not only is learning how to design algorithms a challenging task, but many students believe that algorithm design is not an important part of problem solving. To address this challenge and hopefully change student attitudes, we developed AlgoTutor, a web-based algorithm development tutoring system. AlgoTutor's primary components are the algorithm composer and the algorithm tracer. A third component, ProgramPad, was added to show the connection between algorithms and code. This paper presents the results of experiments that assessed AlgoTutor's effectiveness in changing student attitudes about algorithm development. The results show that students who used AlgoTutor in CS-I were more likely to realize the importance of algorithm design in problem solving and to have confidence in their own algorithm development abilities.

Categories and Subject Descriptors
K.3.1 [**Computer Uses in Education**]: Computer-assisted instruction, Computer-managed instruction, Online learning.

General Terms
Design, Management, Human Factors.

Keywords
algorithm development, learning environment.

1. INTRODUCTION
In beginning computer science classes we introduce "algorithm development prior to coding" as an essential task for problem solving. We teach what is involved in the algorithm development stage in classroom lectures as well as in computer laboratories. We emphasize the importance of algorithm development, and students are frequently required to include an algorithm for programming assignments. However, we have observed that many students ignore this step and fail to acquire algorithm development skills. We think that this is due to two reasons: lack of appreciation of the worth of algorithm development and lack of ability to construct algorithms [9]. Students in introductory computer science classes often feel that there is no need to develop an algorithm for problem solving. This attitude prevents them from building a habit of developing algorithms prior to coding and from acquiring algorithm development skills. There has been a lot of interest in engaging students in programming activities by using visual systems such as Alice or Scratch [2,3,7]. However, there is no tool for practicing algorithm development as a prerequisite of coding as well as helping them learn algorithm development skills.

We have developed an algorithm development tutor (AlgoTutor) for Computer Science (CS) I and II classes [6] [9]. AlgoTutor is a Web-based tutoring system that enables introductory computer-programming students to build algorithms using the top-down design strategy with abstraction refinement. The system can also help students use the bottom-up approach by reusing the algorithms created previously. However, problem solving using a computer programming language such as C++ involves several steps. AlgoTutor also addresses issues that arise in these steps (e.g., verifying the correctness of an algorithm or converting an algorithm to code). In this paper we will show that using AlgoTutor helps students build confidence in algorithm development and recognize the value of algorithm development in problem solving.

In the following section, we discuss steps involved in problem solving using a computer programming language as well as issues related to these steps. In Section 3, we describe the components of AlgoTutor including the algorithm composer, the algorithm tracer, and ProgramPad which automatically generates C++ code from an algorithm developed in the composer. Section 3 also describes the experiments conducted along with the results and lessons learned. Section 4 reports the results of student surveys conducted to assess the effectiveness of AlgoTutor and the students' attitude toward algorithm development. We conclude with future directions of the research.

2. PROBLEM SOLVING PHASES
Problem solving using a computer programming language such as C++ is a complex task which involves several steps:

step 1. Understand the problem specification.
step 2. Develop an algorithm for the problem.
step 3. Verify the correctness of the algorithm by tracing each step of the algorithm.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
SIGCSE'11, March 9–12, 2011, Dallas, Texas, USA.
Copyright 2011 ACM 978-1-4503-0500-6/11/03...$10.00.

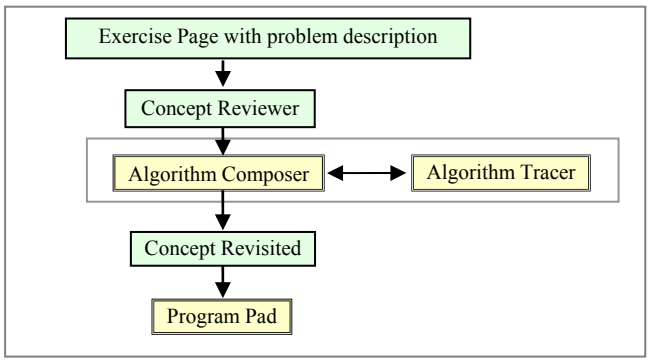


Figure 1. Components of the AlgoTutor system.

step 4. Convert the algorithm into a C++ program.
step 5. Verify whether the converted program is syntactically correct.
step 6. Test and debug the program's correctness.

Steps 5 and 6 are done with the help of compilers and debuggers; however, steps 1 through 4 are typically done manually. In this paper, we describe the system that we have developed to help students with these manual steps (see Figure 1).

In step 1, students need to understand the problem specification by reading the problem description. Additionally, any skills and/or concepts required to solve this problem have to be reviewed/acquired. AlgoTutor uses the concept reviewer component to help students review concepts required and clarify the problem specifications.

The algorithm composer which is the main component of AlgoTutor can help students develop an algorithm for step 2 [6]. Algorithms are constructed by assembling instructor defined pseudo code (pre-defined operations) and control structures via a graphical drag-and-drop interface. The automatic assessment tool in AlgoTutor uses the instructor's solution(s) to give the student online feedback.

For step 3, we have developed an algorithm tracer that can be used for a certain category of problems (usually simple, small problems). The algorithm tracer allows students to verify the correctness of their algorithm by providing visual tracing of the student algorithm.

Once an algorithm has been developed for the problem, students need to figure out what kinds of programming skills are necessary to convert the algorithm to a program. AlgoTutor uses the concept revisited component to help students review the required programming concepts.

For step 4, we have developed another component of AlgoTutor called ProgramPad. ProgramPad converts most of the operations/steps in the student developed algorithm into C++ code, and provides an interface for completing the program. Using ProgramPad, the student can then see the connections between the algorithm that is developed using the composer and the program that is implemented from the algorithm. In Section 3, the algorithm development process using each of the components of AlgoTutor is described.

3. ALGORITHM DEVELOPMENT

The AlgoTutor system has been developed to address the undesirable attitude that developing algorithms prior to writing code is not necessary [9]. In Section 4.2, we will show evidence that students who used AlgoTutor in CS-I appreciate the usefulness of algorithm development and are more confident in developing algorithms. But in this section we will describe each of the components of AlgoTutor along with some experiences we have had with our students.

3.1 Concept Reviewer and Concept Revisited

AlgoTutor starts the tutoring session by providing a problem description as an exercise in a lab. Before students start constructing an algorithm, AlgoTutor uses the concept reviewer component to help students review concepts required to solve the problem and clarify the problem specifications. For example in solving the binary search problem, we asked students to figure out the correct search sequence given the key and the array values. Additionally, we asked students to identify the condition that indicates the situation when the key is not in the array. After the algorithm development phase is completed, AlgoTutor uses the concept revisited component to help students review the required programming skills.

Questions used in these components are presented online in multiple choice or short answer format, and the student responses are graded with appropriate feedback. We have used these two components to evaluate understanding of the problem specification and programming skills as well as for assessment of student learning.

3.2 Algorithm Composer

The Algorithm Composer is the core component of AlgoTutor. This component allows algorithms to be constructed by assembling pre-defined operations and control structures via a graphical drag-and-drop interface. AlgoTuor divides the algorithm design process into two phases. The first phase called the operation selection phase involves knowing which operations to use and in what order to use them. In the second phase called the parameter mapping phase, the student must map the variables in the generic operations to problem specific values.

Figure 2 shows a snapshot of the algorithm composer that solves the problem of drawing a triangle using a nested loop. The available operations and variables are both displayed in the right pane using tabs. A working algorithm under development is displayed in the left pane, which is composed by dragging operation tiles and variable tiles from the right pane [6].

The AlgoTutor system provides online grading and feedback so that students can check immediately whether or not their algorithm is correct and what mistakes, if any, they made. When the "Compare my ordering w/ teacher's" button is clicked, the system will highlight the last correct operation tile in blue and the first incorrect operation in red so that a student can fix the error and complete the algorithm. If they run into a problem that cannot be resolved on their own, students can use a lifeline which will provide the next correct step.

3.3 Algorithm Tracer

In a typical CS-I course much of the teaching effort is expended in teaching correct syntax and use of the programming

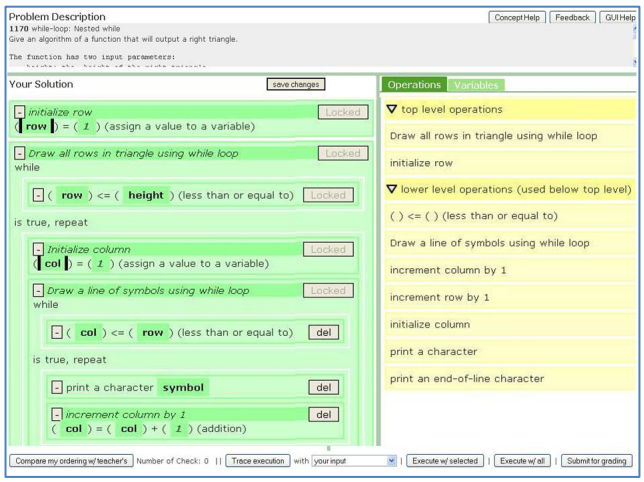


Figure 2. A snapshot of the algorithm composer that solves the problem of drawing a triangle using a nested loop.

environment. The high overhead of learning syntax and a programming environment can distract students from focusing on the primary goal, i.e., learning the process of designing an algorithm for problem solving. Shackelford [8] observed that the details of a particular programming language tended to "annoy and distract attention from the core issue of algorithmic problem solving." Furthermore, even when they learn to construct algorithms on paper, novice students are typically unable to follow the execution of the algorithm to check for its correctness. Algorithm Tracer addresses these issues.

To run the algorithm tracer, the student first selects a test input from the combo box located adjacent to the "Trace execution" button in the algorithm composer, as shown in Figure 2. A trace may require values for input problem parameters and/or the input stream, the student can provide their own values or use one of the test data sets provided by teachers. The algorithm tracer visualizes the execution of the algorithm by stepping one operation or control structure part at a time. The student can follow the control flow and monitor changes in variable values.

Figure 3 shows a snapshot of the algorithm tracer for the algorithm shown in Figure 2. The tracer interface is divided into four panes: the algorithm pane, the variable pane, the output pane and the control panel. The algorithm pane shows a listing of the current algorithm in a format similar to that of the composer. Operations are numbered for reference during execution of the algorithm. The next operation to be executed is highlighted as shown in Figure 3 (step 8). The variable pane shows a listing of all currently defined local variables with their current values. The last operation step that modified the value of that variable is shown in parenthesis to the right of the value. For input problem parameters, "input" is shown instead. The output pane shows output from the execution as it progresses. The control panel lies along the bottom of the window.

The algorithm tracer allows students to verify the correctness of their algorithm by providing visual tracing of the student algorithm. When a student submits an algorithm for grading, it will be run against a known correct solution for all predefined test cases. The outputs for each test case are compared and the student receives a score that is the percentage of all test cases that matched. These grading runs are performed without visualization,

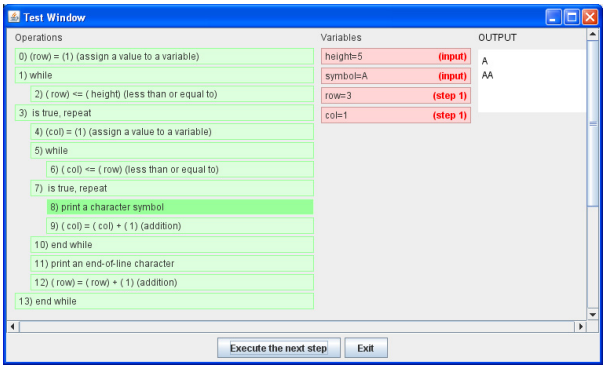


Figure 3. A snapshot of the algorithm tracer for the algorithm in Figure 2.

i.e. no diagnostic information will be provided. Students can also execute the algorithm with selected data to see the output of the execution without tracing by clicking the "Execute w/ selected" button in Composer.

Experiences with Algorithm Tracer

We used AlgoTutor with the algorithm tracer for while-loop labs. Participants in these studies were students enrolled in the CS-I classes in the fall of 2009 and the spring of 2010 at our school. Each of these labs contained four problems. We used pre-tests and post-tests to measure the effectiveness of these labs (and the outcome of the experiments was positive), but we could not isolate the effect of the algorithm tracer from the algorithm composer [10]. To assess the students' perception of the usefulness of the algorithm tracer, an exit survey response was collected online after the second lab. The survey question was "The tracing feature helped me verify my algorithm." A total of 111 students participated in the exit survey, and 77% of the students (85 out of 111) agreed or strongly agreed with the question and only one student disagreed.

The exit survey also asked students to provide any comments related to their experience with the system. The following are typical positive comments provided by students:

- If there happens to be an error, it helps to be able to see where the algorithm goes wrong.
- I really really like tracing.
- The trace function is very helpful. For me, it's the best feature by far. I wish there were more traceable examples, even outside of lab, that I could access for studying and comprehending programs.
- It was very helpful to find out the errors if the algorithm was incorrect

Negative comments were mostly related to very specific situations such as a system glitch which has since been addressed, misunderstanding a problem specification, or inability to solve that specific problem.

Currently the algorithm tracer can only be used for a certain category of problems whose algorithms consist of traceable operations. These operations are implemented so that the execution can be visualized. We are planning to expand the set of traceable operations in the future. In the following subsection, we will describe another component of AlgoTutor called ProgramPad which addresses step 4 of the problem solving process, "convert the algorithm into a C++ program."

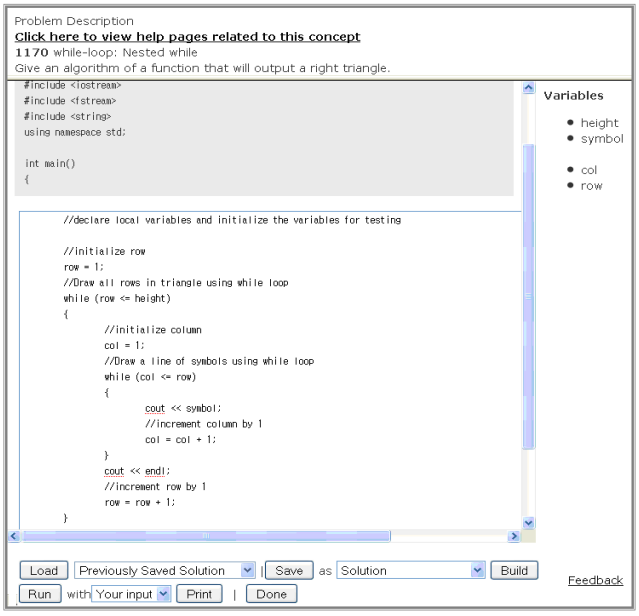


Figure 4. A snapshot of ProgramPad.

3.4 ProgramPad

The main target student group of ProgramPad is students in CS-I classes. The algorithm developed using the composer eventually needs to be converted to code for implementation. Students in CS-I often do not see the relationship between the algorithm and the code. For this reason, we have developed another component of AlgoTutor called ProgramPad. ProgramPad converts most of the operations/steps in an algorithm into C++ code, and provides an interface for completing the program.

Figure 4 shows a snapshot of the program pad for the algorithm that solves the problem of drawing a triangle using a nested loop. ProgramPad displays the problem description in the top pane for students to review the problem specification. The problem parameters and local variables used are displayed in the variable pane on the right. The program pane on the left is used to either convert the student algorithm developed in the composer into C++ code or to load the previously saved code. Along with the capability of converting the algorithm to C++ code, ProgramPad also provides most of a simple integrated development environment's capabilities: edit, save, load, build, run, and print. The control panel displays the buttons related to these features.

ProgramPad can be used in two ways: 1) as the main function that solves the problem or 2) as a function that is called by a driver. Each operation used in AlgoTutor has corresponding C++ code which is either a primitive statement or a function prototype. For example, "print variable" is converted to "cout << variable;" and "print a right triangle of height h with symbol s" is converted to "void PrintTriangle (int h, char s);" An operation with a function prototype is an abstract operation that is not atomic and is viewed as a function in C++. This allows us to convert an algorithm with abstract operations into a program with multiple functions.

Programming concepts related to functions are not easy for novice programmers. They typically have difficulties differentiating between the function parameters and local variables and/or between arguments in a function call and parameters in the corresponding function definition. In addition, students are often confused with the concepts of returning a value from a value-returning function and outputting a value with an output statement. Similarly, they have trouble distinguishing between input-parameters and input statements.

ProgramPad can also be used for more complex problems which contain subproblem(s). For example, the problem of drawing a trapezoid can be solved by using an abstract operation "draw a line of symbols." This operation is a problem which students have already solved. When ProgramPad is invoked for drawing a trapezoid, it automatically provides function prototypes and most of the function definitions which are then automatically converted using the algorithms developed in the algorithm composer as well as the driver that uses these functions.

Repeatedly practicing on problems similar to the above problems using ProgramPad can help students learn the programming concepts while understanding the connection between the algorithm and the corresponding program. Unlike the algorithm composer, ProgramPad does not provide automatic grading. Students can complete and test the program, which has been converted from their algorithm, using their own input values. The program can be printed for grading and/or for help.

Table 1. **:** Students' responses to the survey questions after using ProgramPad (A for agree, N for nuetral, D for disagree in %).

Questions	A	N	D
While concept related			
This exercise helped me learn how to check input errors using a while loop.	79	18	3
Function concept related			
This exercise helped me understand pass-by-value parameter. For example, the value parameter n can be used as a loop control variable.	66	29	5
This exercise helped me understand pass-by-reference parameters. For example, reference parameters can be used to return multiple values.	62	28	11
Top-down/bottom-up strategy related			
This exercise helped me learn to develop a top-level algorithm without worrying about details.	61	33	6
ProgramPad related			
Did ProgramPad help you understand the process of developing a program using the algorithm for this problem?	81	18	2

Experiments with ProgramPad

We used AlgoTutor with ProgramPad for several labs including while-loop, function, and array concepts. Participants in these studies were students enrolled in the CS-I course in the fall of 2009 and the spring of 2010. There were several classes taught by three teachers. To assess the students' perception of the usefulness of the ProgramPad system, exit survey responses were collected online after each problem. Table 1 shows the survey results categorized by concepts that the exercise addressed. Some of the questions are programming concept specific and others are problem solving related or ProgramPad related. Not all exercises were done by all classes, so each question has 22 to 130

responses. The actual survey questions have 5 options: strongly agree, agree, neutral, disagree, and strongly disagree. However, we merged strongly agree and agree as well as strongly disagree and disagree for simplicity.

The data shows that survey results for the while-loop are better than the function. We think it is because the function programming concept is more difficult than the while-loop concept. However, more than 62% of the students agreed for all the questions related to the function concept. For the question related to problem solving strategy, 61% of the students found it useful/helpful. More than 80% of the students agreed that ProgramPad helped them understand the process of developing a program using an algorithm.

4. STUDENT SURVEYS

4.1 How effective is AlgoTutor?

At the end of the 2009 fall and 2010 spring semesters, we also conducted a written exit survey to obtain student feedback for the AlgoTutor system. Table 2 shows a part of the questionnaire used in the exit survey for the algorithm composer and ProgramPad. Questions 1 through 5 refer to the algorithm composer and Questions 6 through 8 are for ProgramPad. Participation in the exit survey was voluntary, and a total of 111 students participated. The student responses are summarized in Table 2. From that data we see that roughly 80% of the students agreed or strongly agreed with "Selecting operations from the provided list helped you figure out the solution," and "The process of choosing parameters helped you figure out the solution." This shows that two phases in the algorithm composer, the operation selection phase and the parameter mapping phase, helped students figure out the solution. We also received favorable responses on ProgramPad.

The exit survey also gave students an opportunity to comment. The following are typical comments provided by students.

Table 2. : A part of the exit survey questionnaire and the survey results. (A for agree, N for nuetral, D for disagree in %).

Survey questions	A	N	D
1. Not worrying about C++ syntax helped you concentrate on problem solving.	71	21	7
2. Selecting operations from the provided list helped you figure out the solution.	79	12	9
3. The process of choosing parameters helped you figure out the solution.	83	12	5
4. The algorithm tutor helped you organize your thoughts and develop an algorithm.	80	10	9
5. The algorithm tutor helped you understand the process of developing an algorithm.	78	13	9
6. Being able to see the C++ code that corresponds to my algorithm helped me understand the algorithm/program.	75	19	6
7. Being able to execute the program for my algorithm and to see the output helped me understand the algorithm	73	21	6
8. The ProgramPad helped you bridge the gap between the algorithm development and the implementation of the algorithm.	59	27	13

Comments on Algorithm Composer

- It really helped me organize my thoughts, and at times it provided an algorithm solution to the one I had in mind.
- We don't have to worry about typos and we can focus on solving the problem rather than stressing out over semi-colons and other syntax errors, and trace execution was very helpful.
- I enjoyed the visual representation of nesting. Nesting was a harder concept to grasp, especially trying to trace execution before using AlgoTutor.
- It developed a foundation to understand how to start analyzing the problem first, than being able to code the program was more clear.

Comments on ProgramPad

- I liked that it helped see how a program is made from an algorithm.
- It was a good way to get real coding experience without having to write the entire program yourself.
- You actually got to see the code and then you got to finish it.
- It did help show the connection between the AlgoTutor and actual coding
- I got to see the algorithm I just made into C++ form

There were some constructive criticisms that will be used to improve the system. Most of the personal conversations with students were very positive and we are quite encouraged to know that students were able to learn a new programming concept and reinforce their learning using the AlgoTutor system.

4.2 Can we change the student's attitude toward algorithm development?

As we mentioned in an earlier section, we observed that students often think developing an algorithm prior to coding is not necessary. They are also lacking algorithm development skills. To find out whether AlgoTutor addresses these two issues, we conducted a survey during the first class of CS-II in the fall of 2010. A total of 52 students participated in this survey. Roughly half of these students used AlgoTutor in CS-I (Group 1), while the other half (Group 2) had not used the system previously.

The survey was designed to assess the students' attitudes toward "lack of appreciation of the worth of the algorithm development phase" and "lack of ability to construct algorithms." Table 3 summarizes the results that were statistically significant.

Specifically, Table 3 shows that more students in group 1 (students who used AlgoTutor in CS-I) responded that they are confident in developing an algorithm than students in group 2 (students who did not use AlgoTutor previously). It also shows that more students in group 1 found that developing an algorithm helped in solving programming tasks than students in group 2. One way ANOVA tests were conducted to determine if there were any significant differences between the two groups and both results are statistically significant with the 95% confidence level.

From the results we reported in this paper, we claim that using AlgoTutor in CS-I helps students:

- Learn proper problem solving processes
- Learn how to develop an algorithm for a problem
- Learn the value of algorithm development
- Develop confidence in algorithm development

Table 3. : Students' responses to the attitude survey questions (percent of students who either agree or strongly agreed).

Questions	Group1	Group2
I know how to develop an algorithm for a programming assignment.	54%	21%
I found it easier to solve a programming task with an algorithm than without it.	63%	29%

5. RELATED WORK

The drag-and-drop features of Alice were a major inspiration in the design of the AlgoTutor interface. While Alice [2] and Scratch [3] [7] allow for the creation of engaging animated content, we used a simpler visualization method for illustrating the algorithms typically encountered in an introductory programming course. The visualization technique provided by ALVIS Live! [4] is at the correct level of abstraction, but it does not provide relief from the burden of language syntax. Even though, there has been some work on automatic code generation from design patterns [1] or an object model like a UML class drawing diagram tool [5] which represents the logical structure of the class templates, none of the systems uses the abstract algorithm to generate code.

While other teaching tools have been developed with similar features, most have been developed as stand-alone applications running on the end user's computer. With AlgoTutor, we chose to create a web-based application that is hosted on a central server and runs within a web browser. This approach allows easy deployment and access from locations outside the lab. Because all data is stored centrally, it is easy to develop tools for instructors and to analyze student performance and behavior.

6. CONCLUSION AND FUTURE WORK

Acquiring the ability to design an algorithm for a problem is one of the most important learning outcomes of computer science courses. Unfortunately, not only is learning how to design algorithms a challenging task, but many students believe that algorithm design is not an important part of problem solving. We developed AlgoTutor to address this challenge and hopefully change student attitudes toward algorithm design.

AlgoTutor trains CS-I students to start the problem solving process with algorithm design. Using the algorithm tracer in AlgoTutor, the system then helps students learn how to trace an algorithm by visualizing the execution of a student algorithm. Using ProgramPad in AlgoTutor, students can see the connection between the algorithm that is developed using the algorithm composer and the converted C++ program. Even difficult programming concepts such as function parameters and array concepts can be addressed using ProgramPad.

Our survey results show that students feel that AlgoTutor helped them organize their thoughts and develop algorithms. But more importantly, students who used AlgoTutor were significantly more likely to appreciate the importance of algorithm design and to have confidence in their ability to develop an algorithm. In the upcoming semesters we plan to run more experiments with new features such as reviewing related concepts and problem specifications. We are also investigating ways to promote student thinking and engaging student interest by adapting to each individual student's learning.

7. ACKNOWLEDGMENTS

This research was supported by the NSF CCLI grant DUE-0737257.

8. REFERENCES

[1] Budinsky, F.J. Finnie, M.A., Vlissides, J.M., and Yu, P.S. (1996). "Automatic Code Generation from Design Patterns" *Object technology*, vol. 35, No. 2.

[2] Cooper, S., Dann, W., & Pausch, R. (2000) Alice: A 3-D tool for introductory programming concepts. In Proceedings of the 5th Annual CCSC Northeastern Conference 2000, Ramapo, NJ.

[3] Fesakis, G. and Serafeim, K. (2009). Influence of the familiarization with "scratch" on future teachers' opinions and attitudes about programming and ICT in education Volume 41 , Issue 3 (September 2009), ITiCSE '09, Pages 258-262.

[4] Hundhausen, C.D. and Brown, J.L. (2007). What You See Is What You Code: A 'Live' Algorithm Development and Visualization Environment for Novice Learners. *Journal of Visual Languages and Computing* 18(1), 22-47.

[5] Niaz, I.A. and Tanaka, J. (2004) "Mapping UML Statecharts to Java Code", in *Proc. IASTED International Conf. on Software Engineering (SE 2004)*, Innsbruck, Austria, pp. 111-116.

[6] Pettey, C., Yoo, J., Yoo, S., Seo, S., Dong, Z., (2009). A Tool for Promoting Algorithm Development in Introductory CS Classes, Proceedings of ED-MEDIA 2009, pp 87-95, Honolulu, Hawaii.

[7] Resnick, M., Maloney, J., Monroy-Hernández, A., Rusk, N., Eastmond, E., Brennan, K., Millner, A., Rosenbaum, E., Silver, J., Silverman, B, and Kafai, Y. (2009). "Scratch: programming for all". Communications of the ACM , Volume 52 Issue 11.

[8] Shackelford, R. L. (1997). "Introducing computer science fundamentals before programming". In FIE '97: Proceedings of the Frontiers in Education Conference, 1997. on 27th Annual Conference. Teaching and Learning in an Era of Change., Washington, DC, USA, pp. 285-289 vol.1. IEEE Computer Society.

[9] Yoo, J., Dong, Z., Yoo, S., Seo, S., and Pettey, C. (2009). "Improving Student Performance by Enforcing Algorithm Development" Proceedings of ED-MEDIA 2009, pp 119-127, Honolulu, Hawaii.

[10] Yoo, J., Pettey, C., Seo, S., and Yoo, S. (2010), Teaching Programming Concepts Using Algorithm Tutor. Proceedings of ED-MEDIA 2010, Toronto, Canada.